

Risk Scoring in Customer 360 Persona Resolution

A methodological note on formulation, calibration, and empirical behavior

Trieu Nguyen

2026-08-04

Abstract

This note formalizes the risk-scoring subsystem used by the Customer 360 Persona Resolution engine. The model maps heterogeneous risk signals into a bounded scalar score on $[0, 100]$ and then discretizes that score into four operational strata: low, medium, high, and critical. The score is defined as an additive composition of churn propensity, domain risk segmentation, and KYC compliance status, with deterministic clipping to preserve codomain invariants. We document the parameterization strategy, threshold semantics, synthetic-data calibration used in development environments, and observed behavior on a seeded portfolio. The objective is to provide a transparent, auditable, and tunable scoring specification that remains computationally simple while preserving operational interpretability.

1. Scope and System Context

The present document covers the risk component of persona computation in the identity-resolution pipeline:

Master Profile -> Persona Engine -> Risk Score -> Risk Level -> Persona Persistence

At runtime, the risk score contributes both directly and indirectly:

1. Directly, as `risk_score` in persona records for governance and monitoring.
2. Indirectly, via inverse contribution ($100 - \text{risk_score}$) in the composite persona score.

Implementation artifacts referenced in this note:

- Risk computation logic: `backend-system/identity_resolution/identity_resolution/persona_engine.py`
- Demo data calibration: `backend-system/identity_resolution/scripts/seed_full_demo_data.py`
- Persistence schema: `database-init/database-schema.sql`

2. Formal Definition of Risk Score

2.1 Base Equation

The engine computes risk as:

$$\text{risk_score} = \text{clip}_{[0,100]} (\alpha \cdot p_{\text{churn}} + b_{\text{segment}} + b_{\text{kyc}})$$

where:

- $p_{\text{churn}} \in [0, 1]$ is churn probability from the master profile.
- $\alpha = 100.0$ maps probability space to score space.
- b_{segment} is a categorical bonus derived from `risk_segment`.
- b_{kyc} is a categorical bonus derived from `kyc_status`.
- $\text{clip}_{[0,100]}(x) = \min(100, \max(0, x))$.

If churn probability is missing, a deterministic fallback base is used:

$$\alpha \cdot p_{\text{churn}} \leftarrow 20.0$$

This preserves total computability for incomplete profiles.

2.2 Bonus Functions

Risk-segment bonuses:

Segment	Bonus
low	0
medium	15
high	30
critical	45

KYC-status bonuses:

KYC status	Bonus
verified	0
pending	10
unverified	20
rejected	40

These two terms encode orthogonal business semantics: operational/regulatory context (`risk_segment`) and identity-compliance certainty (`kyc_status`).

2.3 Worked Examples

1. $p_{\text{churn}} = 0.74$, segment=high, kyc=verified:

$$74 + 30 + 0 = 104 \Rightarrow 100$$

2. $p_{\text{churn}} = 0.68$, segment=medium, kyc=unverified:

$$68 + 15 + 20 = 103 \Rightarrow 100$$

3. $p_{\text{churn}} = 0.32$, segment=low, kyc=verified:

$$32 + 0 + 0 = 32$$

3. Risk-Level Discretization

The numeric score is mapped to categorical levels through fixed thresholds:

$$\text{risk_level}(s) = \begin{cases} \text{critical,} & s \geq 80 \\ \text{high,} & 60 \leq s < 80 \\ \text{medium,} & 40 \leq s < 60 \\ \text{low,} & s < 40 \end{cases}$$

Threshold constants:

Constant	Value
RISK_LEVEL_CRITICAL_THRESHOLD	80.0
RISK_LEVEL_HIGH_THRESHOLD	60.0
RISK_LEVEL_MEDIUM_THRESHOLD	40.0

This discretization is intentionally monotonic and piecewise-constant, enabling deterministic downstream policy routing.

4. Design Rationale

4.1 Additive Formulation

An additive model is preferred for three reasons:

1. Decomposability: each term has clear semantic ownership.
2. Auditability: contributions can be reported without surrogate explanation models.
3. Operational tuning: bonus terms and thresholds can be adjusted independently.

4.2 Explicit Clipping

Hard clipping to $[0, 100]$ ensures numerical stability and preserves interface contracts for all consumers of persona outputs.

4.3 Separation of Behavioral and Compliance Signals

Churn probability captures behavioral attrition risk, while KYC and risk-segment bonuses encode compliance and domain controls. Keeping them separate avoids conflating model uncertainty with governance status.

5. Calibration in Development Data

The seeded development portfolio is intentionally non-uniform, reflecting long-tail properties observed in real customer distributions.

5.1 Churn Prior

- 65%: uniform on $[0.00, 0.25]$
- 20%: uniform on $[0.25, 0.55]$
- 10%: uniform on $[0.55, 0.80]$
- 5%: uniform on $[0.80, 1.00]$

5.2 KYC Prior

- 70% verified
- 15% pending
- 10% unverified
- 5% rejected

5.3 Risk-Segment Prior

- 60% low
- 30% medium
- 10% high

These priors prevent pathological synthetic portfolios where extreme-risk categories are unrealistically overrepresented.

6. Empirical Behavior on Seeded Portfolio

After reset-and-seed execution, the observed persona risk distribution is:

Risk level	Count	Share	Mean score	Range
critical	44	2.6%	90.71	80.7-100.0
high	60	3.5%	69.68	60.3-79.6
medium	115	6.8%	48.77	40.2-59.6
low	1481	87.1%	18.86	0.0-39.7
total	1700	100%	-	-

Interpretation:

1. Category mass is concentrated in low risk, as intended by priors.

2. Inter-bin separation is preserved; score ranges align with threshold boundaries.
3. High and critical tails remain small enough for operational triage.

7. Integration with Composite Persona Score

Risk enters the composite persona score through inversion:

$$\text{persona_score} = \dots + w_{\text{risk}} \cdot (100 - \text{risk_score})$$

with default $w_{\text{risk}} = 0.15$. This design penalizes elevated risk without dominating behavior, engagement, or value signals.

8. Parameter Governance and Change Protocol

All relevant numeric parameters are centralized in `backend-system/identity_resolution/identity_resolution/persona_engine.py`, enabling controlled modifications.

Recommended change protocol:

1. Modify constants only (no inline literals in business logic).
2. Update threshold-focused tests in `backend-system/identity_resolution/tests/test_persona_engine.py`.
3. Re-seed and recompute full distribution to detect category drift.
4. Record pre/post distributions for audit and rollback readiness.

Minimal verification commands:

```
cd backend-system/identity_resolution
.venv/bin/python -m pytest tests/test_persona_engine.py -k risk_level -v
./run_tests.sh
cd ../../
./dev-start-all.sh reset -y
```

9. Limitations and Future Work

Current limitations:

1. Linear additive form may underfit nonlinear interactions (e.g., joint effects of high churn and rejected KYC).
2. Fixed thresholds may drift under temporal portfolio shifts.
3. Bonus schedules are rule-based and require periodic governance review.

Potential extensions:

1. Quantile-based adaptive thresholds with backtesting safeguards.
2. Segment-conditional thresholding by tenant/domain.
3. Post-hoc calibration (e.g., isotonic/Platt-style) when empirical miscalibration is observed.

10. Conclusion

The current risk-scoring design offers a pragmatic balance among interpretability, auditability, and operational usefulness. Its explicit parameterization, bounded codomain, and deterministic tiering make it suitable for enterprise deployment where transparency and policy traceability are first-order requirements. The documented calibration strategy and validation loop provide a reproducible path for controlled evolution as portfolio behavior changes over time.